Assignment 2: Transformer

INFO7375 Semantic Intelligence in LLM, Fall 2026

Fufan Lu (lu.fu@northeastern.edu)

Problem 1: Byte-Pair Encoding (BPE) Tokenizer

Q1.1 A. Failures of a public tokenizer

All three tests use `google-bert/bert-base-cased` (WordPiece); the code is in `tests/test_tokenizer.py` and all three assertions pass.

- **Not injective.** `"I love` *<robot emoji U+1F916>*`"` and `"I love` *<parrot emoji U+1F99C>*`"` both encode to `[146, 1567, 100]`, because every character outside the WordPiece vocabulary becomes the same `[UNK]` token (id 100). Two different strings therefore share one token sequence.
- **Not invertible.** The tokenizer splits punctuation into its own token and `decode` joins tokens with spaces, so `"don't stop"` comes back as `"don ' t stop"`. The original string cannot be recovered from its tokens.
- **Does not preserve concatenation.** `tokenize("token"+"ization")` is `[token, ##ization]`, while `tokenize("token") + tokenize("ization")` is `[token] + [i, ##zation]`. Inside a word WordPiece uses `##` continuation pieces, which differ from the pieces used at the start of a word.

Q1.1 B. Why non-trivial tokenizers cannot preserve concatenation

Suppose a tokenizer preserves concatenation, and let $t \in \mathcal{V}$ be any multi-character token, $t = xy$ with x, y non-empty strings. Then $\text{tokenize}(t) = \text{tokenize}(x) + \text{tokenize}(y)$, and since both parts are non-empty token sequences, t is tokenized into at least two tokens: the single token t can never be emitted. Applying the same argument to every string shows that $\text{tokenize}(c_1 \cdots c_n) = \text{tokenize}(c_1) + \cdots + \text{tokenize}(c_n)$, i.e. the tokenizer is completely determined by how it maps individual characters, so every multi-character token in the vocabulary is unreachable and the tokenizer is equivalent to the trivial one. Put differently, the whole point of merging (e.g. `a`, `b` $\rightarrow$ `ab`) is to tokenize a string differently depending on its neighbours, and concatenation creates exactly such new neighbours at the boundary.

Q1.2 BPE implementation

`ASCIIBPETokenizer.merge`, `encode` and `decode` are implemented in `src/tokenizer/bpe.py`; all `test_*_merge`, `test_encode` and `test_decode` tests pass. `encode` replays the learned merges in the order they were learned; one pass suffices because a later merge never creates a new occurrence of an earlier bigram.

Q1.3 Learned tokens of the English tokenizer

A. The longest learned token is `"\n\nReferences"` (12 bytes), followed by its intermediate pieces `"ferences"` and `"feren"` and then frequent words such as `" which"`, `" with"`, `" from"`, `" that"`. This shows the training data is Wikipedia: nearly every article ends with a *References* section, so that boilerplate string was frequent enough to be merged into a single token.

B. BPE merges whatever strings are most frequent in the training corpus, so a sensitive string that is repeated often (a name, email address, phone number or API key) can become a token, and publishing the vocabulary then reveals that the string was in the private data. The merge order also leaks relative frequencies, which can be used to infer the composition of the training data.

Q1.4 English vs. Thai

A. Yes: the English document uses 119 tokens while its Thai translation uses 636 (5.3×), although the two strings have a similar number of characters (249 vs. 239). The tokenizer was trained only on English, so it learned no merges for Thai; every Thai character takes 3 bytes in UTF-8, and the 659 bytes end up as 636 tokens, almost one token per byte.

B. Users of low-resource languages pay more for the same content (APIs bill per token), fit less text into the context window, wait longer for generation, and usually get worse model quality because each token carries less information, which is a systematic unfairness between languages.

Problem 2: Implementing a Transformer

Q2.1 Weight tying

Weight tying (Press & Wolf, 2017) uses one matrix both as the input embedding and as the output projection, so the logit of token v is the dot product of the final hidden state with v's input embedding. This removes the separate $V \times d$ output matrix, which is a large share of a small model's parameters (here $V = 50257$), and acts as a regularizer. The paper shows that the tied embedding, trained through both its input and output roles, gives lower perplexity.

Q2.2 Logits with weight tying

```
logits = h @ E.T # (b, s, d) @ (d, v) -> (b, s, v); same as torch.einsum("bsd,vd->bsv", h
    , E)
```

Q2.3 Implementation

`MultiHeadAttention`, `FeedForward`, `DecoderBlock` and `DecoderLM` are implemented in `src/lm/model.py` without loops over matrix operations; all tests in `tests/test_model.py` pass (the CUDA-only test is skipped on a machine without CUDA). Design choices: pre-LN blocks (GPT-2 style, matching the final `LayerNorm` in `DecoderLM`); masked logits are filled with the dtype minimum rather than $-\infty$ and the masked weights are zeroed after the softmax, so rows that are entirely padding output zeros instead of NaN; position ids are `cumsum(mask) - 1` so left padding does not shift positions; logits use weight tying.

Problem 3: Training the Transformer

Q3.1 Implementation

All functions in `src/lm/train.py` are implemented and all tests in `tests/test_train.py` pass, including `test_train_sanity_check` (validation loss $2.080 \approx \log 8$ in five repeated runs, i.e. the model does not see the token it predicts). The training loop clips the gradient norm at 1.0 before each optimizer step.

Q3.2 Cosine annealing with warmup

A.

$$\mathrm{lr}(t) = \begin{cases} \mathrm{lr}_{max}\,\dfrac{t}{a}, & 0 \le t < a \quad \text{(warmup)} \\ \mathrm{lr}_{min} + \dfrac{1}{2}\left(\mathrm{lr}_{max} - \mathrm{lr}_{min}\right)\left(1 + \cos\left(\pi\,\dfrac{t-a}{b-a}\right)\right), & a \le t < b \quad \text{(annealing)} \\ \mathrm{lr}_{min}, & t \ge b \end{cases}$$

B. Large steps early make fast progress while the model is far from a good solution, and steadily smaller steps later let the optimizer settle into a low-loss region instead of bouncing around it, so cosine annealing usually reaches a lower final loss than a constant learning rate; the schedule is smooth with no decay milestones to tune, and the short warmup avoids unstable first updates while Adam's moment estimates are still poor.

Q3.3 GPT-tiny

A. With the provided `GPT-tiny.yaml` (2000 steps, 8.13×10^{13} FLOPs) the validation loss is **7.052** (perplexity 1155).

B. The provided 2000 steps are far from saturation: the loss is still falling when the schedule ends. To find where it saturates I trained the same configuration for longer (only `num_training_steps` changed):

steps	tokens	val loss	val PPL
2,000	8.2M	7.052	1155
8,000	32.8M	6.269	528
20,000	81.9M	6.046	422

The training loss flattens after roughly **10,000 steps** (≈40M tokens): it is about 6.35 at step 10k and only about 6.3 at step 20k (Figure 1), while quadrupling the steps from 2k to 8k had gained 0.78 nats of validation loss. So GPT-tiny should be trained for about 10k steps; beyond that its 2M parameters (mostly the embedding, $d = 32$) leave little to gain.

C. Training curve of the provided GPT-tiny run (logged to Weights & Biases as `train-loss`; the dashed line is the learning rate):

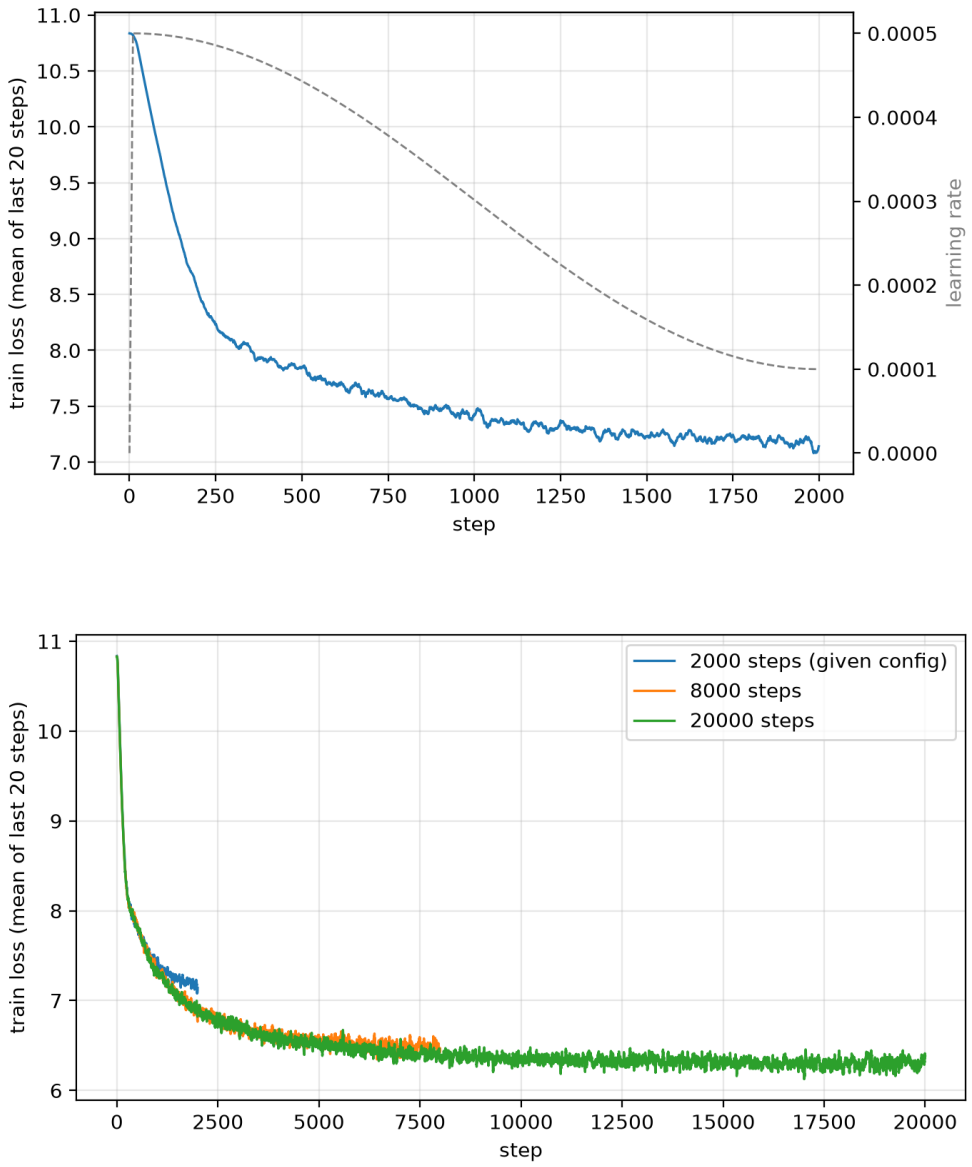


Figure 1: GPT-tiny trained for 2k, 8k and 20k steps (each with its own cosine schedule).

Q3.4 Hyperparameter search under 10^{15} FLOPs

A. Procedure. Every run uses at most 10^{15} training FLOPs as counted by `train.py`; for each configuration I set `num_training_steps` $= \lfloor 10^{15}/(\text{FLOPs per token} \times \text{batch} \times \text{seq_len}) \rfloor$. Because Q3.5 keeps the model hyperparameters fixed and must reach perplexity below 50 with 10^{17} FLOPs and a checkpoint under 100MB, I limited the search to models that can plausibly do that: with the 50257-token vocabulary, the tied embedding alone holds $50257\,d$ parameters, so an fp32 checkpoint under 100MB requires $d \leq 320$; I therefore considered $d \in \{256, 320\}$ with 6–12 layers (17–24M parameters), keeping `n_positions` $=$ `seq_len` $= 128$. The search had two rounds with selection rules fixed in advance (validation perplexity on the held-out 5M-token split): round 1 kept the model fixed (d256, 8 layers, 8 heads) and varied the training recipe; round 2 applied the best recipe

to a deeper (12 layers) and a wider ($d = 320$, 6 layers, head size 64) model. All five runs used one NVIDIA H200 with bf16 autocast. As a check, configuration A was also trained on a V100 in fp32 and reached perplexity 407.2 (vs. 402.6), so the numeric precision barely matters.

B. Results.

	what changed	params	steps	tokens	val loss	val PPL
A	d256/L8/H8; GPT-tiny recipe: lr 5e−4→1e−4, warmup 10, batch 32	19.2M	2061	8.4M	5.998	402.6
B	**as A, lr 2e−3→2e−4, warmup 100**	19.2M	2061	8.4M	**5.845**	**345.4**
C	as B, batch 16 (twice the updates, same tokens)	19.2M	4122	8.4M	5.949	383.4
D	as B, 12 layers (deeper)	22.4M	1756	7.2M	5.952	384.7
E	as B, $d = 320$, 6 layers, 5 heads (wider)	23.5M	1694	6.9M	5.914	370.3

The learning rate mattered most: raising the peak learning rate from 5×10^{-4} to 2×10^{-3} lowered perplexity from 402.6 to 345.4. Halving the batch size to get twice as many updates hurt (383.4), presumably because the noisier gradients at this learning rate outweigh the extra steps. Under a fixed FLOP budget, the larger models see fewer tokens (7.2M and 6.9M instead of 8.4M) and end up worse even though the wider model E learns fastest per token early on (Figure 2); at 10^{15} FLOPs the budget is too small for the extra capacity to pay off. The best configuration is **B**: $d = 256$, 8 layers, 8 heads, 128 positions (19.2M parameters, 76.9MB), peak learning rate 2×10^{-3}.

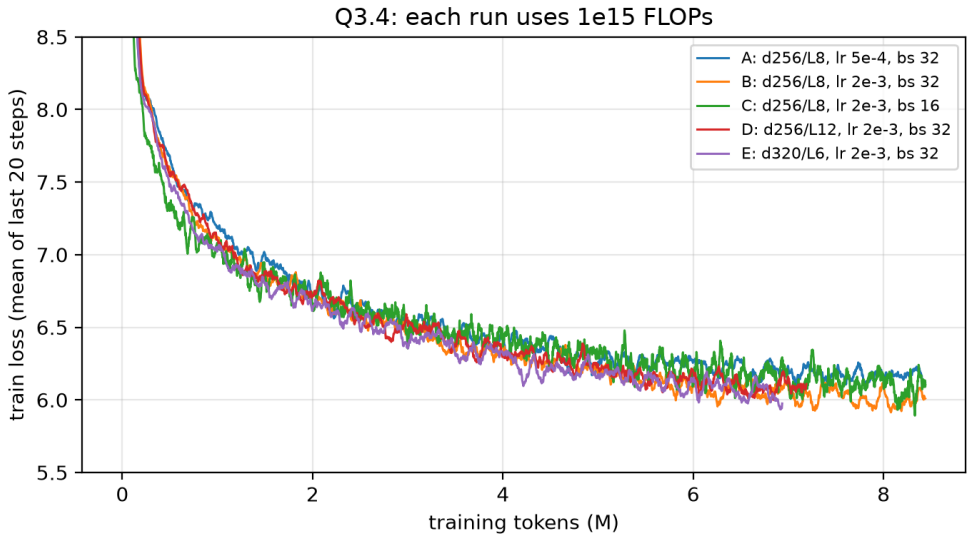


Figure 2: Training loss of the five configurations against training tokens.

C. Final configuration (`configs/hparam/B_d256L8_lr2e-3.yaml`):

```
output_dir: outputs/hparam/B_d256L8_lr2e-3
tokenizer_encoding: gpt2
model_config:
  n_embd: 256
  n_head: 8
  n_positions: 128
  n_layer: 8
device: auto
batch_size: 32
```

```
seq_len: 128
num_warmup_steps: 100
num_training_steps: 2061
grad_accumulation_steps: 1
min_lr: 0.0002
max_lr: 0.002
```

Q3.5 Final model

The final model keeps the model hyperparameters of configuration B ($d = 256$, 8 heads, 128 positions, 8 layers; 19.2M parameters) and only changes the training hyperparameters to use $\approx 10^{17}$ FLOPs (1.00×10^{17} as reported by `train.py`): batch size 256×128 tokens, 25,764 steps (≈844M tokens, about 1.6 passes over the 520M training tokens), 515 warmup steps (2%), peak learning rate 2×10^{-3} annealed to 2×10^{-4}. A larger batch than in Q3.4 keeps the GPU busy (39 TFLOPS on one H200, 45 minutes).

Final validation loss	**3.596**
Final validation perplexity	**36.46**
Checkpoint (`model.pt`)	76.9MB

Configuration (`config.yaml`, submitted with `model.pt`):

```
output_dir: outputs/final
tokenizer_encoding: gpt2
model_config:
  n_embd: 256
  n_head: 8
  n_positions: 128
  n_layer: 8
device: auto
batch_size: 256
seq_len: 128
num_warmup_steps: 515
num_training_steps: 25764
grad_accumulation_steps: 1
min_lr: 0.0002
max_lr: 0.002
```

Problem 4: Text Generation & Perplexity Analysis

Q4.1 Implementation

`softmax_with_temperature` and a batched `generate` (left padding with `eot_token`, attention mask 0 on padding) are implemented in `src/lm/generate.py`; `test_generate.py` passes. The reported perplexity is computed over the generated tokens only: exp of the mean negative log-likelihood the model assigns (at temperature 1) to each sampled token given everything before it.

Q4.2 Generations

A. Prefixes and settings. I used three of the provided prefixes (`data/prefixes.jsonl`: a news-magazine paragraph, a 19th-century biography and a popular-science news lead):

1. Hammer a tube called a spile into the bark of a maple tree and its translucent yellow sap will start flowing out; the tree has been tapped (pictured). This gloop drips from some 55m Canadian maple trees today, raw material for the tangy golden syrup which is one of the country's proudest exports.
2. CHARLES H. MEISNER, one of the prosperous and highly-esteemed business men of Birnamwood, Shawano county, was born in Newton township, Manitowoc Co., Wis., February 14, 1860, and is a son of Frederick and Magarette (Steltzer) Meisner, both natives of Germany.
3. EVER WONDER WHY SOME PEOPLE CAN WORK NIGHTS AND SLEEP EASILY DURING THE DAY? A new study from Tiina Pauno, a professor at the University of Helsinki, Finland found that many night workers actually have a variant in the genetic code that makes them less influenced by Melatonin and hence the circadian rhythm.

Both models generated `max_new_tokens` $= 48$ tokens per prefix at temperatures $T \in \{0.01, 0.5, 1.0\}$ (our model with `src/lm/generate.py` and the Q3.5 checkpoint; Pythia-1.4B with `src/lm/use_pythia.py`, which samples with `do_sample=True`). $T = 0.01$ stands in for "near zero" because Hugging Face sampling requires $T > 0$. Generations, prefix by prefix:

Our model (final checkpoint from Q3.5) *Prefix 1.*

- $T = 0.01$: The tree is a small, small
- $T = 0.5$: The color is a plumber, but it is a great choice for the grain and the taste. It was a great time to see the best of the very best of the new and the most brilliant ingredients to have. The fruit
- $T = 1.0$: This structure is unplugged and makes it HUD at least the most popular ever-designed magazine around the world before. ANoticeless ascended rather than even writing a second and gieser guild to buy groceries into the duckwall to invest

Prefix 2.

- $T = 0.01$: The first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first of the first
- $T = 0.5$: The company also has been considered as a brand new company. The company has been considered as a brand new company, a company founded in 1986, and is a member of the company. The company has been considered as a brand
- $T = 1.0$: /? Others from this site Reading to 6 Constitution Required: Since 2001, the literary community will be presented primarily from the original Xbox One Company in Virginia, the American Indian Book". (1) Obama Leader Essays to

Prefix 3.

- $T = 0.01$: The study was conducted by the University of Helsinki, Finland, which was conducted by the University of Helsinki, Finland, and the University of Helsinki, Finland. The study was conducted by the University of Helsinki, Finland, and the University of
- $T = 0.5$: The study was conducted by a professor at the University of Helsinki, Finland, where she was an independent professor at the University of Helsinki, Finland. The study was conducted by a professor at the University of Helsinki, Finland, and was the
- $T = 1.0$: Read more about Johannes, who did not write this and booth possibly metabolic studies, which included Giendouke Leonard Baker in the research groups of the other studies written in an attempt to leverage the its ownurrence. Jones has gone into this

Pythia-1.4B *Prefix 1.*

- $T = 0.01$: The sap is collected in the spring, when the maple trees are still green, and is then boiled down to make maple syrup. The sap is then collected in the fall, when the maple trees are already brown and the
- $T = 0.5$: The process of tapping maple trees is a complex one, involving a number of steps. The sap rises from the tree's trunk and runs down a tube into a wooden spile, which is then attached to a wooden handle.
- $T = 1.0$: TAP: What is maple syrup? A: It's the sticky, syrup-like sap dripping from the maple tree. B: It's one of Canada's top-ranking agricultural export products. TAP: How is it made (

Prefix 2.

- $T = 0.01$: He was educated in the public schools of Manitowoc county, and at the age of eighteen years was employed as a clerk in the office of the county treasurer. He was married to Miss Mary E. (Hoffman) Me
- $T = 0.5$: He was educated in the public schools of Manitowoc county, and received his education in the public schools and private schools of Wisconsin. He was graduated from the University of Wisconsin at Madison, Wisconsin, in 1882, and from the law department
- $T = 1.0$: His father was born in Gau-Sachsen, Prussia, February 16, 1823, and is a native of Wernigerode. He graduated from the high school at Dresden, Germany, in 1851,

Prefix 3.

- $T = 0.01$: The study, published in the journal Nature Communications, found that the variant is associated with a lower level of melatonin production. The researchers found that the variant is associated with a lower level of melatonin production.
- $T = 0.5$: Researchers analyzed the genetic makeup of more than 3,000 people who worked in factories, offices, and other places where there is a high risk of exposure to chemicals. They found that the genes for Melatonin production were more common
- $T = 1.0$: So do you need sleep and wake up often throughout the day? The answer is most likely no. But some people get used to doing that. They say things they would never say at work, or they go out and drink when

Perplexities of the generated continuations (3 prefixes × 48 tokens) at each temperature. "Self" is each model's perplexity on its own continuations (temperature-1 probabilities of the sampled tokens given everything before them, as printed by `generate.py`); "judged by Pythia" scores the continuations with Pythia-1.4B as a common reference model (`scripts/score_continuations.py`).

T	ours, self	Pythia, self (= judged by Pythia)	ours, judged by Pythia
0.01	2.10	3.33	2.80
0.5	7.21	3.49	16.55
1.0	193.09	7.02	575.94

B. Near zero the softmax turns into an argmax, so decoding becomes greedy and deterministic: both models fall into loops (ours repeats "small, small, small, . . . " and "The first of the first of. . . "; Pythia repeats "the variant is associated with a lower level of melatonin"), and perplexity is very low. Near one the models sample from their full distribution, so the text is more varied and less repetitive but also less coherent, which for our small model means drifting into nonsense (perplexity 193).

C. At $T = 0.01$ our model even scores a lower perplexity than Pythia (2.10, or 2.80 judged by Pythia, vs. 3.33), but only because its degenerate loops are trivially predictable, so a low perplexity

does not mean better text. At $T = 0.5$ and 1.0, Pythia's continuations stay fluent and on topic while ours are grammatical but empty at 0.5 and incoherent at 1.0; Pythia finds our text 5–80× less likely than its own (16.55 vs. 3.49, 575.94 vs. 7.02). The gap comes from scale: Pythia-1.4B has about 70× more parameters, was trained on about 300B tokens of the Pile (vs. our 0.84B) with a 2048-token context (vs. 128), so its next-token distribution is much sharper, whereas ours (validation perplexity 36.5) spreads probability over many tokens that derail the text when sampled.

Q4.3 Perplexity under Pythia-1.4B

Document	PPL (Pythia-1.4B)
Northeastern University Wikipedia paragraphs	7.10
same text with every "the"/"The" removed (8 words)	9.27
Jabberwocky (original)	1.58
Jabberwocky mimic (nonsense words swapped)	18.92

Each document was evaluated on its own (`--batch_size 1`), since `evaluate_perplexity.py` pools all documents in a file into one number.

A. The Wikipedia paragraphs have perplexity 7.10, which rises to 9.27 after every "the" is removed. "the" is one of the easiest tokens to predict, and removing it also makes the text ungrammatical, so the words that follow (e.g. a noun the model expected to come after a determiner) become harder to predict, raising the average negative log-likelihood.

B. The original Jabberwocky has perplexity 1.58, while my mimic has 18.92. The original is a famous poem that almost certainly appears many times in Pythia's training data (*Through the Looking-Glass* is in Project Gutenberg, which is part of the Pile, and the poem is quoted all over the web), so the model has essentially memorized it, nonsense words included; my invented words never appeared in training, so the model cannot recall them and can only predict the ordinary English around them.

My mimic poem (first stanza shown; every stanza swaps several nonsense words, the full text is in `data/ppl/jabberwocky_mimic.jsonl`):

```
'Twas frindle, and the plorky voats
     Did skirl and wumble in the drabe:
All flimty were the kittipoos,
     And the zome grackths outsnabe.
```

Problem 5: Use of Generative AI

Q5.1. Yes: Claude Code (Anthropic's coding agent).

Q5.2. I used Claude Code to read the assignment and starter code, implement the missing functions, run the tests, set up and launch the training jobs on the Explorer cluster, and draft the written answers, which I reviewed. Sample prompts: "Look at this assignment and tell me how to do it."; "Go ahead and do it; you can use the school's GPU cluster."

Q5.3. No other use.